Software Engineering — Assignment #04

AI-assisted evolution of the Smart Campus Room Availability Checker

Student: Baidildanov Almansur · **Date:** 08 November 2026 · **AI assistant used:** Claude (Claude Code CLI, model Claude Opus 5.5)

Repository: `smart-campus-assignment-04` (Git) — initial version **38413e0**, final version **63d03ff** (3 commits). Run: `npm install`, `npm run dev`, `npm test`, `npm run build`.

Task #01 — Codebase analysis

Starting point. The Smart Campus prototype from Assignment #03 / Practice #08 (React + TypeScript + Vite + Tailwind): one room B-204, fictional booking 600–660, fixed clock now = 540 (09:00). Everything lived in one file, `src/App.tsx` (243 lines). Baseline recorded first: 13 UI characterization tests U01–U13 → **13/13 passed** (`evidence/test_output/01_baseline_tests.txt`).

Context given to the AI (`evidence/ai_runs/prompt_task1_analysis.md`): repository map, AC1–AC6, new requirements F1–F5, constraints (no login/payments/APIs/DB), "read only, do not edit, small plan, not a rewrite". The AI ran read-only (Read/Glob/Grep allowed; Edit/Write/Bash disallowed).

Maintainability risks (verified by me in the code):

#	Risk	Evidence (baseline file:line)
R1	Room id "B-204" hard-coded in rule messages and UI — wrong for any other room	`src/App.tsx:53, 63, 136, 142` (`checkAvailability`, header, `InfoRow`)
R2	Two sources of truth for "blocked" (fixture + UI state merged)	`src/App.tsx:76, 106, 120` (`DEMO_ROOM`, `useState`, `handleSubmit`)
R3	One file mixes types, rules, formatting, fixture, parsing and UI; nothing exported → rules testable only through UI	`src/App.tsx:12–243`
R4	Duplicated logic: start/end input markup copied, `parseInt`/NaN copied, booking range formatting copied	`src/App.tsx:154–185`, `116–119`, `59` & `124`
R5	Result computed only on submit → stale banner when room changes (blocks F5)	`src/App.tsx:107, 120`

Prioritized plan (agreed before editing): 0) keep U01–U13 green as safety net → 1) extract pure `domain/` modules, room passed as parameter, clock separate → 2) `data/rooms.ts` catalog (≥4 frozen rooms, B-204 kept identical) → 3) pure `filterRooms` (F1, F2) → 4) split UI into components, empty state (F4), refresh on room switch (F5) → 5) add unit + UI tests.

AI suggestions I rejected / modified:

- **Rejected — "tighten parsing with `/^\d+$/`".** It changes old behaviour (`"660.5"` is accepted today as 660) and is not part of the new requirements; the task says preserve old behaviour. Kept `parseInt` (now in one helper `parseMinutes`) and recorded it as a remaining limitation.

- **Modified — "drop the demo blocked checkbox, take blocked only from room data".** Dropping it would break U07. I kept the checkbox as a demo *override* and compute `blocked = room.blocked || override`, so a blocked catalog room (E-120) can never be available (F3) and U07 still passes.

Task #02 — Feature extension

- **Catalog of 5 fictional rooms** (`src/data/rooms.ts`), each with id, name, capacity, equipment list, blocked flag and **its own bookings**; deep-frozen (AC5).

Room	Name	Capacity	Equipment	Blocked	Bookings
B-204	Seminar Room	30	projector, whiteboard	no	600–660
A-101	Lecture Hall	120	projector, microphone, whiteboard	no	720–840
C-310	Computer Lab	24	computers, projector	no	660–720
D-015	Meeting Room	8	video conferencing, whiteboard	no	—
E-120	Media Studio	12	microphone, video conferencing, projector	**yes**	900–960

- **Filters:** attendees (F1, capacity ≥ attendees) and required-equipment checkboxes (F2, room must have ALL). Only matching rooms are displayed with "N of 5 rooms match".
- **F3:** blocked rooms are shown with a "Blocked" badge but the check always returns *Unavailable*.
- **F4:** clear empty state "No rooms match your requirements" and the check button is disabled.
- **F5:** switching rooms re-runs the existing check against the new room's own bookings; if a filter removes the selected room, the first match is selected and the stale result is hidden.
- Original time / 120-minute / overlap validation unchanged (U01–U13 pass unchanged).

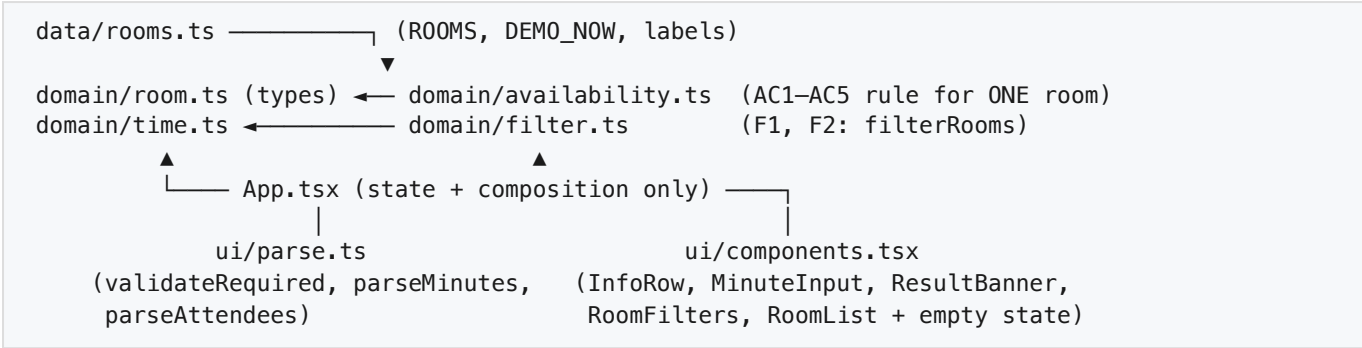
Demonstrated by automated UI tests (actual results): successful filter (I01, I02), unsuccessful filter/empty state (I03), room switching 660–720: B-204 *Available* → C-310 *Overlaps 660–720* → D-015 *Available* (I04), blocked room (I05), stale-result protection (I06).

Task #03 — Refactoring

Before (38413e0):

```
src/App.tsx  (243 lines)
 ├─ types Booking / RoomState / Result
 ├─ checkAvailability()  ← rules + "B-204" text
 ├─ formatTime(), DEMO_ROOM fixture
 ├─ validateRequired(), parseInt ×2
 └─ App UI: 2 copied inputs, inline result banner
```

After (63d03ff):



Dependencies point one way: UI → domain/data; `domain/` has no React and no I/O.

Design decision 1 — pure functions with the room passed in (`checkAvailability(room, now, start, end)`, `filterRooms(rooms, criteria)`). *Problem:* the rule read one hard-coded room. *Benefit:* the check provably uses the chosen room's bookings (test A08, I04), messages use `room.id`, and rules are unit-tested without React.

Design decision 2 — frozen data module + derived selection instead of copied state. The catalog is deep-frozen and the visible list / selected room are *computed* from filters on each render, not stored twice. *Benefit:* no mutation (AC5, tests A07 and F-immutability) and no out-of-sync state when filters remove the selected room (I06).

Not used: Strategy/Adapter/class hierarchies — only one policy and one data shape exist, so they would add complexity without benefit.

Task #04 — Automated testing

Command: `npm test` (Vitest 2.1.2, jsdom). **Result: 5 files, 37 tests, 37 passed, 0 failed** (`evidence/test_output/02_final_tests.txt`). `npm run build` (tsc + vite) passed.

File	Tests	Covers
`tests/ui.regression.test.tsx`	13 (U01–U13)	original AC1–AC6 through the UI, unchanged since baseline
`src/domain/availability.test.ts`	8 (A01–A08)	AC1–AC5; **120 vs 121**; **touching vs 1-minute overlap**; immutability; chosen-room bookings
`src/domain/filter.test.ts`	8 (C01, F1a–F4)	catalog fields; **capacity 30 exact vs 31 insufficient**; ALL equipment; blocked never available; empty result
`src/ui/parse.test.ts`	1 (P01)	required fields and input parsing
`tests/rooms.ui.test.tsx`	7 (I01–I07)	**UI integration**: filters, empty state, room switching F5, blocked room, stale result

Task #05 — Quality evaluation

Measure	Original (38413e0)	Final (63d03ff)	How measured
Automated tests passed / total	13 / 13 (UI only)	**37 / 37**	`npm test` output files 01 and 02

Measure	Original (38413e0)	Final (63d03ff)	How measured
Duplicated logic locations removed	—	**4 removed**	`git show 38413e0:src/App.tsx | grep -c`: "B-204" ×4 → 0 in rules/UI (uses `room.id`); start/end input markup ×2 → `MinuteInput`; `parseInt` +NaN ×2 → `parseMinutes`; booking range formatting ×2 → `rangeMinutes/rangeClock`
Code structure	1 module, 243 lines, 6 responsibilities	8 modules: data (30), domain 4 files (97), ui 2 files (165), App (138)	`wc -l`
Rooms supported	1	5 (filters F1–F5)	`src/data/rooms.ts`, test C01

Remaining limitations: `parseInt` still accepts `"660.5"` as 660 (kept for backward compatibility); the "Room blocked (demo)" checkbox is a global demo override; fixed demo clock (540); tests run in jsdom, not on a real phone/browser; no screen-reader audit.

Task #06 — AI reflection

I used Claude (Claude Code CLI, Claude Opus 5.5). I gave it a concise repository map, the six acceptance criteria, the new F1–F5 requirements and the constraints, and ran it read-only so it could analyse but not change code. Its analysis was useful: it correctly found the hard-coded "B-204" messages, the double source of truth for the blocked flag, the stale result that blocks F5 and the single-file structure, with exact line references that I checked in the file. Two suggestions were not taken as given. Tightening number parsing would silently change existing behaviour, so I rejected it and listed it as a limitation. Removing the demo blocked checkbox would break the old U07 test, so I modified it into an override combined with the room's own flag. I personally verified the line references, ran the baseline tests before any edit, read every diff, and checked boundary values myself: 120 vs 121 minutes, a booking that only touches versus overlaps by one minute, and 30 versus 31 attendees for a 30-seat room. Tests drove the decisions: keeping U01–U13 unchanged forced me to keep message texts, labels and the default room identical, and the integration test for room switching showed that a result must be tied to the room it was computed for, which led to hiding stale results after filtering. Next I would make number parsing strict as a separate, tested change, replace the global demo override with per-room data only, and add a real-browser test on desktop and mobile widths.

Appendix — AI evidence / AI-use record

Tool & date	Prompt summary	Output used	Problems found	Changes made	Verification
Claude Code CLI, Claude Opus 5.5, 08.11.2026 (session 605c3670)	Repo map + AC1–AC6 + F1–F5 + constraints; “read-only, list ≥3 risks with file:line, small prioritized plan”	6 risks, 7-step plan (`response_task1_analysis.md`)	Parsing change would alter old behaviour; dropping checkbox would break U07	Rejected parsing change; modified checkbox into override	Line refs checked in `38413e0:src/App.tsx`; U01–U13 13/13 before and after
Claude Code CLI, Claude Opus 5.5, 08.11.2026	Implement plan step by step: modules, catalog, filters, tests	Code in commits fb83042, 63d03ff	First banner version added hidden text → 12 UI tests failed	Removed hidden text, kept `aria-label`	`npm test` 37/37, `tsc` clean, `npm run build` OK

Raw logs: `evidence/ai_runs/raw_task1_analysis.jsonl`, prompt and extracted response in the same folder; test logs in `evidence/test_output/`.